

ADMET Multi-Task Learning — Analiza Eksperymentów

Cel: Zbudowanie modelu MTL, który konsekwentnie przewyższa Single-Task Learning (STL) na 13 zadaniach ADMET.

Każdy eksperyment to jedna konkretna modyfikacja bazowego modelu (`EXPERIMENT_MAIN`).

```
In [1]: import os, warnings
warnings.filterwarnings('ignore')
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from IPython.display import display

plt.rcParams.update({'figure.dpi': 130, 'font.size': 11, 'axes.titlesize': 13})

# Ścieżka do EXPERIMENT/
# Notebook leży w EXPERIMENT/, więc BASE = folder notebooka
BASE = os.path.dirname(os.path.abspath('__file__'))
if not os.path.basename(BASE) == 'EXPERIMENT':
    BASE = os.path.join(os.getcwd(), 'EXPERIMENT')
# Przy nbconvert cwd może być inny – użyj ścieżki bezwzględnej
BASE = '/Users/jqbpoz/Projects/ADMET_MULTI_TASK_LEARNING/EXPERIMENT'
print(f'BASE: {BASE}')
```

BASE: /Users/jqbpoz/Projects/ADMET_MULTI_TASK_LEARNING/EXPERIMENT

1. Opis eksperymentów — rodzaj modyfikacji względem MAIN

Eksperyment	Typ modyfikacji	Co się zmienia	Hipoteza
MAIN	Bazowy	GIN + MaskedBCELoss	Punkt odniesienia
BALANCING	Funkcja kosztu	pos_weight = neg/pos per task	Nierówne klasy — faworyzuje mniejszościową

Eksperyment	Typ modyfikacji	Co się zmienia	Hipoteza
UNCERTAINTY	Funkcja kosztu	Uczone skalary σ^2 per task	Trudne taski dostają mniejszą wagę
CROSS_UNCERT	Funkcja kosztu	Macierz 13x13 $\sigma[i,j]$ — cross-task	Task i może wyciszyć task j jeśli kolidują
PCGRAD	Optymalizacja	Gradient Surgery — projekcja kolidujących gradientów	Eliminuje destruktywne interakcje gradientów
GROUPING	Architektura	Dwa backbone GIN: CYP-cluster vs reszta	Rozdziela gradienty na poziomie sieci
EDGE_FEAT	Architektura	GINEConv + 6 cech wiązań (typ, pierścień, konjugacja)	Dodatkowa informacja chemiczna
VIRTUAL_NODE	Preprocessing	Wirtualny węzeł — globalny przepływ w 1 przeskoku	Pokonuje ograniczony zasięg 3-warstwowego GIN

Zadania ADMET (13 zbiorów)

Klaster	Zadania
CYP (metabolizm)	cyp2c19, cyp2d6, cyp3a4, cyp1a2, cyp2c9
Transport/ADME	hia_hou, pgp_broccatelli, bbb_martins
Toksyczność	herg, dill, skin_reaction, ames, clintox

```
In [2]: VARIANT = 'GNN_RDKIT_MORGAN'

EXPERIMENTS = {
    'MAIN': ('EXPERIMENT_MAIN', 'MTL_GNN'),
    'BALANCING': ('EXPERIMENT_BALANCING', 'MTL_GNN_BALANCED'),
    'UNCERTAINTY': ('EXPERIMENT_UNCERTAINTY', 'MTL_GNN_UNCERTAINTY'),
    'CROSS_UNCERT': ('EXPERIMENT_CROSS_UNCERTAINTY', 'MTL_GNN_CROSS'),
    'PCGRAD': ('EXPERIMENT_PCGRAD', 'MTL_GNN_PCGRAD'),
    'GROUPING': ('EXPERIMENT_GROUPING', 'MTL_GNN_GROUPED'),
    'EDGE_FEAT': ('EXPERIMENT_EDGE_FEATURES', 'MTL_GNN_EDGE'),
    'VIRTUAL_NODE': ('EXPERIMENT_VIRTUAL_NODE', 'MTL_GNN_VN'),
}

METHOD_COLORS = {
    'STL_GNN': '#95a5a6',
    'MAIN': '#2c3e50',
}
```

```

    'BALANCING':      '#e67e22',
    'UNCERTAINTY':    '#9b59b6',
    'CROSS_UNCERT':   '#8e44ad',
    'PCGRAD':         '#27ae60',
    'GROUPING':       '#2980b9',
    'EDGE_FEAT':      '#c0392b',
    'VIRTUAL_NODE':   '#16a085',
}

series = {}
for label, (exp_dir, col) in EXPERIMENTS.items():
    path = os.path.join(BASE, exp_dir, VARIANT, 'final_results.csv')
    if os.path.exists(path):
        df = pd.read_csv(path).set_index('Task')
        if col in df.columns:
            series[label] = df[col].rename(label)

main_df = pd.read_csv(os.path.join(BASE, 'EXPERIMENT_MAIN', VARIANT, 'final_results.csv')).set_index('Task')
stl = main_df['STL_GNN'].rename('STL_GNN')
xgb = main_df['XGBoost'].rename('XGBoost') if 'XGBoost' in main_df.columns else None

mtl = pd.concat(series.values(), axis=1)
# XGB dodany do full – bierze udział we wszystkich heatmapach i rankingach
# XGB dostępny osobno do per-task porównań, NIE w full (avg XGB jest mylące)
full = pd.concat([stl, mtl], axis=1)
METHOD_COLORS['XGBoost'] = '#f39c12'
TASKS = list(full.index)

TASK_CLUSTERS = {
    'CYP':      [t for t in TASKS if 'cyp' in t],
    'Transport': ['hia_hou', 'pgp_broccatelli', 'bbb_martins'],
    'Tox':      ['herg', 'dili', 'skin_reaction', 'ames', 'clintox'],
}

print(f'Wczytano {len(series)} eksperymentow, {len(TASKS)} zadan')
print(full.round(4).to_string())

```

Wczytano 8 eksperymentow, 13 zadan

	STL_GNN	MAIN	BALANCING	UNCERTAINTY	CROSS_UNCERT	PCGRAD	GROUPING	EDGE_FEAT	VIRTUAL_NODE
Task:									
hia_hou	0.9826	0.9810	0.9669	0.9864	0.9788	0.9902	0.9696	0.9805	0.9957
pgp_broccatelli	0.9046	0.9344	0.9107	0.9140	0.9218	0.9115	0.9356	0.9061	0.9257
bbb_martins	0.8650	0.8769	0.8724	0.8689	0.8739	0.8861	0.8688	0.8520	0.8610
cyp2c19_veith	0.8846	0.9073	0.8844	0.8748	0.8810	0.8984	0.9070	0.8996	0.8803
cyp2d6_veith	0.8403	0.8691	0.8533	0.8376	0.8474	0.8661	0.8671	0.8652	0.8395
cyp3a4_veith	0.8892	0.8984	0.8802	0.8685	0.8789	0.8951	0.9036	0.8976	0.8741
cyp1a2_veith	0.9261	0.9386	0.9204	0.9171	0.9207	0.9337	0.9388	0.9358	0.9262
cyp2c9_veith	0.8770	0.9071	0.8863	0.8707	0.8808	0.8993	0.9113	0.9077	0.8906
herg	0.8069	0.8186	0.7973	0.7611	0.8653	0.8214	0.8304	0.8097	0.8041
dili	0.8218	0.8457	0.7877	0.8719	0.8710	0.8568	0.8320	0.8351	0.8918
skin_reaction	0.6346	0.7124	0.7812	0.7188	0.6932	0.7360	0.6926	0.7079	0.6767
ames	0.8570	0.8789	0.8600	0.8487	0.8512	0.8812	0.8743	0.8739	0.8432
clintox	0.9066	0.9040	0.9238	0.9192	0.9233	0.9478	0.8780	0.8887	0.9037

2. Ranking ogólny — średni AUROC

```
In [3]: means = full.mean()
main_col = full['MAIN']
delta_main = full.subtract(main_col, axis=0).mean()
wins = (full.subtract(main_col, axis=0) > 0).sum()
losses = (full.subtract(main_col, axis=0) < 0).sum()
beats_stl = (full.subtract(full['STL_GNN'], axis=0) > 0).sum()
ranking = pd.DataFrame({
    'Sr. AUROC': means.round(4),
    'vs MAIN [pp]': (delta_main * 100).round(2),
    'Wins vs MAIN': wins,
    'Losses vs MAIN': losses,
    'Bije STL (z 13)': beats_stl,
}).sort_values('Sr. AUROC', ascending=False)

def color_delta_r(val):
    if isinstance(val, float):
        if val > 0.1: return 'color: #27ae60; font-weight: bold'
        if val < -0.1: return 'color: #c0392b'
    return ''

def color_beats(val):
```

```

if not isinstance(val, (int, float)): return ''
if val >= 10: return 'background-color: #27ae6033; font-weight: bold'
if val >= 7: return 'background-color: #f39c1233'
return 'background-color: #c0392b22'

```

```

display(ranking.style
        .format({'Sr. AUROC': '{:.4f}', 'vs MAIN [pp]': '{:+.2f} pp'})
        .applymap(color_delta_r, subset=['vs MAIN [pp]'])
        .applymap(color_beats, subset=['Bije STL (z 13)'])
        .background_gradient(subset=['Sr. AUROC'], cmap='YlGn')
        .set_caption(f'Ranking metod MTL + STL — wariant {VARIANT}'))

```

Ranking metod MTL + STL — wariant GNN_RDKIT_MORGAN

	Sr. AUROC	vs MAIN [pp]	Wins vs MAIN	Losses vs MAIN	Bije STL (z 13)
PCGRAD	0.8864	+0.39 pp	7	6	13
MAIN	0.8825	+0.00 pp	0	0	11
GROUPING	0.8776	-0.49 pp	5	8	11
CROSS_UNCERT	0.8760	-0.65 pp	3	10	8
EDGE_FEAT	0.8738	-0.87 pp	1	12	10
BALANCING	0.8711	-1.14 pp	2	11	7
VIRTUAL_NODE	0.8702	-1.23 pp	2	11	6
UNCERTAINTY	0.8660	-1.65 pp	4	9	6
STL_GNN	0.8612	-2.12 pp	2	11	0

```

In [4]: fig, ax = plt.subplots(figsize=(11, 5))
sorted_cols = full.mean().sort_values(ascending=False).index
vals = full.mean()[sorted_cols]
colors = [METHOD_COLORS.get(c, '#7f8c8d') for c in sorted_cols]

bars = ax.bar(sorted_cols, vals, color=colors, edgecolor='white', linewidth=0.8, zorder=3)
ax.axhline(full['MAIN'].mean(), color='#2c3e50', linestyle='--', lw=1.5,
            label=f'MAIN ({full["MAIN"].mean():.4f})', zorder=4)
ax.axhline(full['STL_GNN'].mean(), color='#95a5a6', linestyle=':', lw=1.5,

```

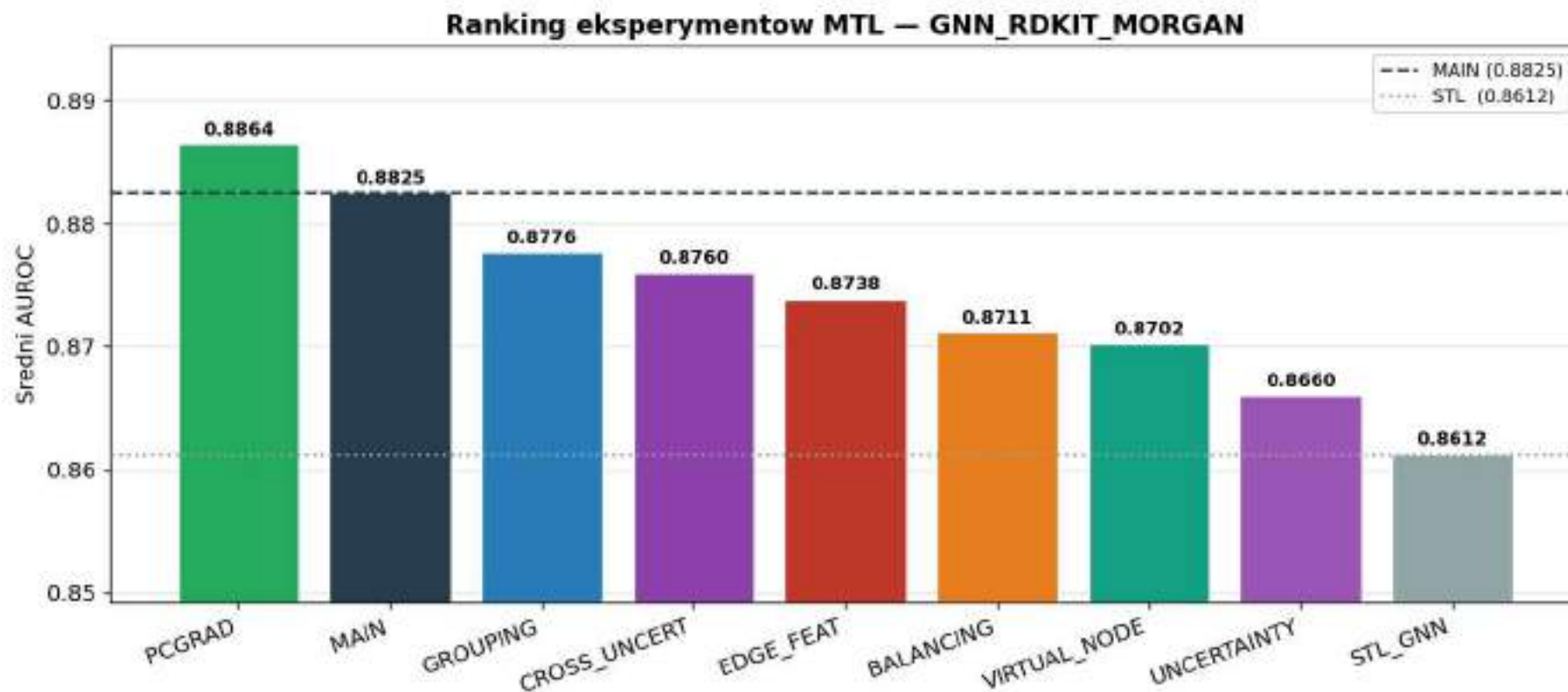
```

label=f'STL ({full["STL_GNN"].mean():.4f})', zorder=4)
# XGBoost nie pokazany na avg – patrz per-task tabele w sekcji 5

for bar, val in zip(bars, vals):
    ax.text(bar.get_x() + bar.get_width()/2, val + 0.0005, f'{val:.4f}',
            ha='center', va='bottom', fontsize=9, fontweight='bold')

ax.set_ylim(min(vals) - 0.012, max(vals) + 0.008)
ax.set_ylabel('Sredni AUROC')
ax.set_title(f'Ranking eksperymentow MTL – {VARIANT}', fontweight='bold')
ax.legend(fontsize=9)
ax.grid(axis='y', alpha=0.3, zorder=0)
ax.set_xticklabels(sorted_cols, rotation=20, ha='right')
plt.tight_layout()
plt.show()

```



3. Heatmapa AUROC — wszystkie metody × wszystkie zadania

```
In [5]: col_order = ['STL_GNN'] + [c for c in full.columns if c != 'STL_GNN']
heat_data = full[col_order].T

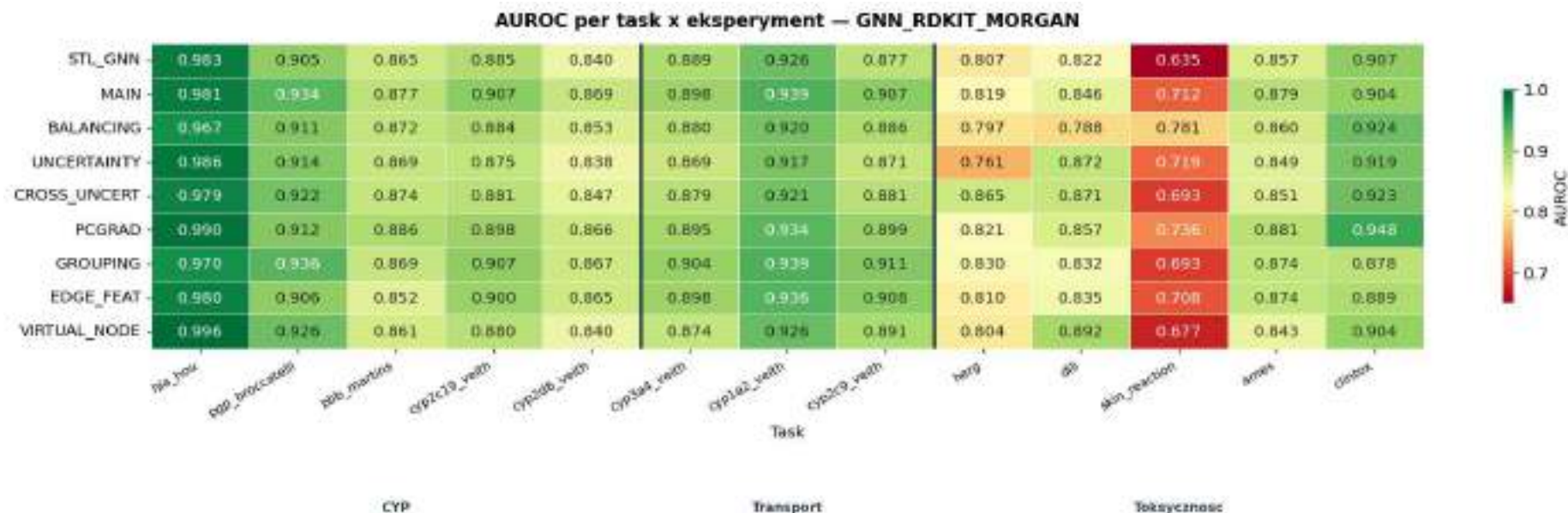
fig, ax = plt.subplots(figsize=(17, 6))
sns.heatmap(heat_data.astype(float), annot=True, fmt='.3f', cmap='RdYlGn',
            linewidths=0.4, linecolor='white', vmin=0.65, vmax=1.0, ax=ax,
            cbar_kws={'label': 'AUROC', 'shrink': 0.7})

for b in [5, 8]:
    ax.axvline(b, color='#2c3e50', linewidth=2.5)

ax.set_title(f'AUROC per task x eksperyment — {VARIANT}', fontweight='bold', pad=12)
ax.set_xticklabels(ax.get_xticklabels(), rotation=30, ha='right', fontsize=9)

for lbl, (s, e) in [('CYP', (0,5)), ('Transport', (5,8)), ('Toksy czność', (8,13))]:
    ax.text((s+e)/2, -0.5, lbl, ha='center', va='top', fontsize=9, fontweight='bold',
            color='#2c3e50', transform=ax.get_xaxis_transform())

plt.tight_layout()
plt.show()
```



4. Delta AUROC vs MAIN — per metoda i zadanie

```
In [6]: methods = [c for c in full.columns if c not in ('STL_GNN', 'MAIN')]
delta_df = full[methods].subtract(full['MAIN'], axis=0) * 100 # pp

# Reorder tasks: CYP | Transport | Tox
task_order = (
    [t for t in TASKS if 'cyp' in t] +
    ['hia_hou', 'pdp_broccatelli', 'bbb_martins'] +
    ['herg', 'dili', 'skin_reaction', 'ames', 'clintox']
)
task_order = [t for t in task_order if t in delta_df.index]
plot_data = delta_df.loc[task_order].T # methods x tasks

fig, ax = plt.subplots(figsize=(16, 5))
sns.heatmap(
    plot_data.astype(float),
    annot=True, fmt='+.1f', cmap='RdYlGn', center=0,
    linewidths=0.5, linecolor='white',
    vmin=-8, vmax=8, ax=ax,
```



```

cbar_kws={'label': 'Δ AUROC vs MAIN [pp]', 'shrink': 0.7},
)

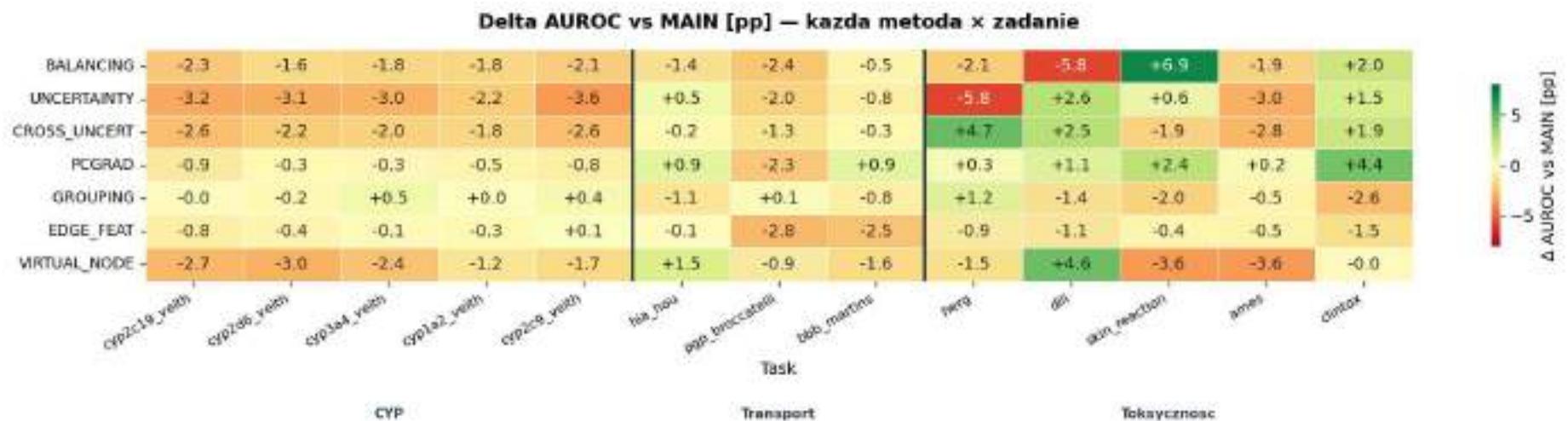
# Cluster separators (columns = tasks)
cyp_count = len([t for t in task_order if 'cyp' in t])
transport_count = 3
ax.axvline(cyp_count, color='#2c3e50', linewidth=2.2)
ax.axvline(cyp_count + transport_count, color='#2c3e50', linewidth=2.2)

ax.set_title('Delta AUROC vs MAIN [pp] — kazda metoda x zadanie', fontweight='bold', pad=14)
ax.set_xticklabels(ax.get_xticklabels(), rotation=30, ha='right', fontsize=9)
ax.set_yticklabels(ax.get_yticklabels(), rotation=0, fontsize=10)

for lbl, (s, e) in [('CYP', (0, cyp_count)),
                   ('Transport', (cyp_count, cyp_count+transport_count)),
                   ('Toksyycznosc', (cyp_count+transport_count, len(task_order)))]:
    ax.text((s+e)/2, -0.55, lbl, ha='center', va='top', fontsize=9, fontweight='bold',
           color='#2c3e50', transform=ax.get_xaxis_transform())

plt.tight_layout()
plt.show()

```



5. Najlepsza metoda per zadanie

```

In [7]: # — Szukamy globalnego najlepszego: wszystkie eksperymenty x wszystkie warianty —
ALL_VARIANTS = ['GNN', 'GNN_MORGAN', 'GNN_RDKIT', 'GNN_RDKIT_MORGAN']

ADMET_GROUPS = {
    'Absorption': ['hia_hou', 'pgp_broccatelli', 'bbb_martins'],
    'Metabolism': ['cyp2c19_veith', 'cyp2d6_veith', 'cyp3a4_veith', 'cyp1a2_veith', 'cyp2c9_veith'],
    'Toxicity': ['herg', 'dili', 'skin_reaction', 'ames', 'clintox'],
}
task_to_group = {t: g for g, ts in ADMET_GROUPS.items() for t in ts}

# Zbierz WSZYSTKIE wyniki (exp x variant)
records = []
for exp_label, (exp_dir, col) in EXPERIMENTS.items():
    for var in ALL_VARIANTS:
        p = os.path.join(BASE, exp_dir, var, 'final_results.csv')
        if not os.path.exists(p): continue
        df = pd.read_csv(p).set_index('Task')
        if col not in df.columns: continue
        for task, val in df[col].items():
            if not np.isnan(val):
                records.append({'Task': task, 'Metoda': exp_label, 'Wariant': var, 'AUROC': val})

all_res = pd.DataFrame(records)
# Referencja musi być zdefiniowana przed global_best
main_ref = pd.read_csv(os.path.join(BASE, 'EXPERIMENT_MAIN', 'GNN_RDKIT_MORGAN', 'final_results.csv')).set_index('Task')
stl_ref = main_ref['STL_GNN']
xgb_ref = main_ref['XGBoost'] if 'XGBoost' in main_ref.columns else None
main_mtl = main_ref['MTL_GNN']

global_best = all_res.loc[all_res.groupby('Task')['AUROC'].idxmax()].set_index('Task')

# (main_ref, stl_ref, xgb_ref, main_mtl już zdefiniowane powyżej)

row = {
    'ADMET': pd.Series({t: task_to_group.get(t, '?') for t in global_best.index}),
    'XGBoost': xgb_ref.round(4) if xgb_ref is not None else float('nan'),
    'STL_GNN': stl_ref.round(4),
    'MAIN': main_mtl.round(4),
    'Najlepsza metoda': global_best['Metoda'].reindex(stl_ref.index),
    'Wariant wejsc': global_best['Wariant'].reindex(stl_ref.index),
}

```

```

    'Najlepszy AUROC': global_best['AUROC'].reindex(stl_ref.index).round(4),
    'vs MAIN [pp]': ((global_best['AUROC'].reindex(stl_ref.index) - main_mtl) * 100).round(2),
    'vs STL [pp]': ((global_best['AUROC'].reindex(stl_ref.index) - stl_ref) * 100).round(2),
    'MTL > STL': (global_best['AUROC'].reindex(stl_ref.index) > stl_ref).map({True: 'YES', False: 'NO'}),
}
if xgb_ref is not None:
    gb_auroc = global_best['AUROC'].reindex(xgb_ref.index)
    row['vs XGB [pp]'] = ((gb_auroc - xgb_ref) * 100).round(2)
    row['MTL > XGB'] = (gb_auroc > xgb_ref).map({True: 'YES', False: 'NO'})

per_task_global = pd.DataFrame(row)

def color_bool2(val):
    return 'color: #27ae60; font-weight: bold' if val == 'YES' else 'color: #c0392b; font-weight: bold'

def color_delta2(val):
    if isinstance(val, float):
        if val > 0.5: return 'color: #27ae60; font-weight: bold'
        if val < -0.5: return 'color: #c0392b'
    return ''

def color_group(val):
    p = {'Absorption': '#dce9f7', 'Metabolism': '#fef3cd', 'Toxicity': '#fde8e8'}
    return f'background-color: {p.get(val, "")}; font-weight: bold'

def color_variant(val):
    vc = {'GNN': '#d5e8d4', 'GNN_MORGAN': '#ffe6cc', 'GNN_RDKIT': '#e1d5e7', 'GNN_RDKIT_MORGAN': '#d5f5e3'}
    return f'background-color: {vc.get(val, "")}'

fmt = {
    'XGBoost': '{:.4f}', 'STL_GNN': '{:.4f}', 'MAIN': '{:.4f}', 'Najlepszy AUROC': '{:.4f}',
    'vs MAIN [pp]': '{:+.2f} pp', 'vs STL [pp]': '{:+.2f} pp',
}

bool_cols = ['MTL > STL']
delta_cols = ['vs MAIN [pp]', 'vs STL [pp]']
if xgb_ref is not None:
    fmt['vs XGB [pp]'] = '{:+.2f} pp'
    bool_cols.append('MTL > XGB')
    delta_cols.append('vs XGB [pp]')

display(per_task_global.style

```

```
.format(fmt)
.applymap(color_bool2, subset=bool_cols)
.applymap(color_delta2, subset=delta_cols)
.background_gradient(subset=['Najlepszy AUROC'], cmap='YlGn')
.set_caption('GLOBALNIE – Najlepszy wynik MTL per zadanie (wszystkie metody × wszystkie warianty wejść)'))

n_stl = (global_best['AUROC'].reindex(stl_ref.index) > stl_ref).sum()
n_xgb = (global_best['AUROC'].reindex(xgb_ref.index) > xgb_ref).sum() if xgb_ref is not None else '?'
print(f'\nNajlepsza MTL bije STL na {n_stl}/13 zadaniach')
if xgb_ref is not None:
    print(f'Najlepsza MTL bije XGB na {n_xgb}/13 zadaniach')
```

GLOBALNIE — Najlepszy wynik MTL per zadanie (wszystkie metody × wszystkie warianty wejść)

	ADMET	XGBoost	STL_GNN	MAIN	Najlepsza metoda	Wariant wejść	Najlepszy AUROC	vs MAIN [pp]	vs STL [pp]	MTL > STL	vs XGB [pp]	MTL > XGB
ames	Toxicity	0.8930	0.8570	0.8789	GROUPING	GNN_RDKIT	0.8883	+0.94 pp	+3.13 pp	YES	-0.48 pp	NO
bbb_martins	Absorption	0.9008	0.8650	0.8769	VIRTUAL_NODE	GNN_RDKIT	0.9187	+4.18 pp	+5.37 pp	YES	+1.79 pp	YES
clintox	Toxicity	0.9226	0.9066	0.9040	PCGRAD	GNN_RDKIT_MORGAN	0.9478	+4.39 pp	+4.12 pp	YES	+2.53 pp	YES
cyp1a2_veith	Metabolism	0.9345	0.9261	0.9386	EDGE_FEAT	GNN	0.9417	+0.31 pp	+1.56 pp	YES	+0.73 pp	YES
cyp2c19_veith	Metabolism	0.8913	0.8846	0.9073	BALANCING	GNN_MORGAN	0.9103	+0.30 pp	+2.57 pp	YES	+1.90 pp	YES
cyp2c9_veith	Metabolism	0.8986	0.8770	0.9071	GROUPING	GNN_RDKIT_MORGAN	0.9113	+0.41 pp	+3.42 pp	YES	+1.27 pp	YES
cyp2d6_veith	Metabolism	0.8654	0.8403	0.8691	EDGE_FEAT	GNN	0.8715	+0.24 pp	+3.12 pp	YES	+0.61 pp	YES
cyp3a4_veith	Metabolism	0.8889	0.8892	0.8984	GROUPING	GNN_RDKIT_MORGAN	0.9036	+0.52 pp	+1.44 pp	YES	+1.47 pp	YES
dili	Toxicity	0.8648	0.8218	0.8457	VIRTUAL_NODE	GNN_RDKIT	0.9207	+7.49 pp	+9.88 pp	YES	+5.59 pp	YES
herg	Toxicity	0.8162	0.8069	0.8186	UNCERTAINTY	GNN_RDKIT	0.8839	+6.52 pp	+7.70 pp	YES	+6.77 pp	YES
hia_hou	Absorption	0.9864	0.9826	0.9810	VIRTUAL_NODE	GNN_RDKIT_MORGAN	0.9957	+1.47 pp	+1.30 pp	YES	+0.92 pp	YES
pgp_broccatelli	Absorption	0.9342	0.9046	0.9344	GROUPING	GNN_RDKIT_MORGAN	0.9356	+0.12 pp	+3.10 pp	YES	+0.14 pp	YES
skin_reaction	Toxicity	0.7551	0.6346	0.7124	BALANCING	GNN_RDKIT_MORGAN	0.7812	+6.89 pp	+14.67 pp	YES	+2.61 pp	YES

Najlepsza MTL bije STL na 13/13 zadaniach
Najlepsza MTL bije XGB na 12/13 zadaniach

5b. Najlepsza metoda per zadanie — per wariant wejść

```
In [8]: VARIANTS = ['GNN', 'GNN_MORGAN', 'GNN_RDKIT', 'GNN_RDKIT_MORGAN']

for var in VARIANTS:
    # Wczytaj wszystkie eksperymenty dla tego wariantu
    s = {}
    for label, (exp_dir, col) in EXPERIMENTS.items():
        p = os.path.join(BASE, exp_dir, var, 'final_results.csv')
        if os.path.exists(p):
            df = pd.read_csv(p).set_index('Task')
            if col in df.columns:
                s[label] = df[col].rename(label)

    if not s:
        continue

    f_var = pd.concat(s.values(), axis=1)

    # STL i XGBoost z MAIN dla tego wariantu
    main_p = os.path.join(BASE, 'EXPERIMENT_MAIN', var, 'final_results.csv')
    if not os.path.exists(main_p):
        continue
    main_v = pd.read_csv(main_p).set_index('Task')
    stl_v = main_v['STL_GNN']
    xgb_v = main_v['XGBoost'] if 'XGBoost' in main_v.columns else None

    bm = f_var.idxmax(axis=1)
    bs = f_var.max(axis=1)

    row = {
        'XGBoost': xgb_v.round(4) if xgb_v is not None else float('nan'),
        'STL_GNN': stl_v.round(4),
        'MAIN': f_var['MAIN'].round(4) if 'MAIN' in f_var.columns else float('nan'),
```

```

    'Najlepsza metoda':      bm,
    'Najlepszy AUROC':      bs.round(4),
    'Delta vs MAIN [pp]':   ((bs - f_var['MAIN']) * 100).round(2) if 'MAIN' in f_var.columns else float('nan'),
    'Delta vs STL [pp]':    ((bs - stl_v) * 100).round(2),
    'MTL bije STL?':       (bs > stl_v).map({True: 'TAK', False: 'NIE'}),
}
if xgb_v is not None:
    row['Delta vs XGB [pp]'] = ((bs - xgb_v) * 100).round(2)
    row['MTL bije XGB?']    = (bs > xgb_v).map({True: 'TAK', False: 'NIE'})

tbl = pd.DataFrame(row)

fmt = {
    'XGBoost': '{:.4f}', 'STL_GNN': '{:.4f}', 'MAIN': '{:.4f}',
    'Najlepszy AUROC':   '{:.4f}',
    'Delta vs MAIN [pp]': '{:+.2f} pp',
    'Delta vs STL [pp]':  '{:+.2f} pp',
}
bool_cols = ['MTL bije STL?']
delta_cols = ['Delta vs MAIN [pp]', 'Delta vs STL [pp]']
if xgb_v is not None:
    fmt['Delta vs XGB [pp]'] = '{:+.2f} pp'
    bool_cols.append('MTL bije XGB?')
    delta_cols.append('Delta vs XGB [pp]')

def color_bool(val):
    return 'color: #27ae60; font-weight: bold' if val == 'TAK' else 'color: #c0392b'

def color_delta(val):
    if isinstance(val, float):
        if val > 0.5: return 'color: #27ae60; font-weight: bold'
        if val < -0.5: return 'color: #c0392b'
    return ''

n_xgb = (bs > xgb_v).sum() if xgb_v is not None else '?'
n_stl = (bs > stl_v).sum()

display(tbl.style
    .format(fmt)
    .applymap(color_bool, subset=bool_cols)
    .applymap(color_delta, subset=delta_cols))

```

```
.background_gradient(subset=['Najlepszy AUROC'], cmap='YlGn')
.set_caption(f'Wariant: {var} | Bije STL: {n_stl}/13 | Bije XGB: {n_xgb}/13'))
print()
```

Wariant: GNN Bije STL: 12/13 Bije XGB: 12/13										
	XGBoost	STL_GNN	MAIN	Najlepsza metoda	Najlepszy AUROC	Delta vs MAIN [pp]	Delta vs STL [pp]	MTL bije STL?	Delta vs XGB [pp]	MTL bije XGB?
Task										
hia_hou	0.9137	0.9680	0.9848	PCGRAD	0.9935	+0.87 pp	+2.55 pp	TAK	+7.98 pp	TAK
pgp_broccatelli	0.8987	0.8975	0.9211	UNCERTAINTY	0.9243	+0.32 pp	+2.68 pp	TAK	+2.55 pp	TAK
bbb_martins	0.8130	0.8742	0.8776	BALANCING	0.8938	+1.62 pp	+1.96 pp	TAK	+8.08 pp	TAK
cyp2c19_veith	0.7727	0.8862	0.8988	GROUPING	0.9079	+0.90 pp	+2.17 pp	TAK	+13.52 pp	TAK
cyp2d6_veith	0.7382	0.8347	0.8694	EDGE_FEAT	0.8715	+0.20 pp	+3.67 pp	TAK	+13.32 pp	TAK
cyp3a4_veith	0.7746	0.8660	0.9022	MAIN	0.9022	+0.00 pp	+3.62 pp	TAK	+12.75 pp	TAK
cyp1a2_veith	0.8464	0.9284	0.9353	EDGE_FEAT	0.9417	+0.64 pp	+1.33 pp	TAK	+9.53 pp	TAK
cyp2c9_veith	0.8045	0.8634	0.9100	GROUPING	0.9106	+0.06 pp	+4.72 pp	TAK	+10.61 pp	TAK
herg	0.7494	0.8626	0.8120	PCGRAD	0.8387	+2.67 pp	-2.38 pp	NIE	+8.94 pp	TAK
dili	0.7952	0.8746	0.8342	VIRTUAL_NODE	0.8945	+6.03 pp	+1.99 pp	TAK	+9.93 pp	TAK
skin_reaction	0.7433	0.7015	0.6741	UNCERTAINTY	0.7398	+6.57 pp	+3.83 pp	TAK	-0.35 pp	NIE
ames	0.7889	0.8646	0.8748	EDGE_FEAT	0.8772	+0.23 pp	+1.26 pp	TAK	+8.83 pp	TAK
clintox	0.8780	0.9141	0.8985	PCGRAD	0.9362	+3.77 pp	+2.21 pp	TAK	+5.82 pp	TAK

Wariant: GNN_MORGAN Bije STL: 13/13 Bije XGB: 11/13										
Task	XGBoost	STL_GNN	MAIN	Najlepsza metoda	Najlepszy AUROC	Delta vs MAIN [pp]	Delta vs STL [pp]	MTL bije STL?	Delta vs XGB [pp]	MTL bije XGB?
hia_hou	0.9300	0.8844	0.9343	BALANCING	0.9555	+2.12 pp	+7.11 pp	TAK	+2.55 pp	TAK
pgp_broccatelli	0.8936	0.9129	0.9077	CROSS_UNCERT	0.9323	+2.46 pp	+1.95 pp	TAK	+3.87 pp	TAK
bbb_martins	0.8652	0.8401	0.8771	EDGE_FEAT	0.8833	+0.62 pp	+4.33 pp	TAK	+1.81 pp	TAK
cyp2c19_veith	0.8621	0.8618	0.8973	BALANCING	0.9103	+1.30 pp	+4.85 pp	TAK	+4.81 pp	TAK
cyp2d6_veith	0.8371	0.8348	0.8621	MAIN	0.8621	+0.00 pp	+2.73 pp	TAK	+2.50 pp	TAK
cyp3a4_veith	0.8584	0.8684	0.8845	PCGRAD	0.8940	+0.95 pp	+2.55 pp	TAK	+3.56 pp	TAK
cyp1a2_veith	0.9044	0.9167	0.9223	EDGE_FEAT	0.9249	+0.25 pp	+0.82 pp	TAK	+2.04 pp	TAK
cyp2c9_veith	0.8652	0.8718	0.9062	CROSS_UNCERT	0.9078	+0.16 pp	+3.60 pp	TAK	+4.26 pp	TAK
herg	0.8576	0.8258	0.8434	UNCERTAINTY	0.8496	+0.62 pp	+2.38 pp	TAK	-0.80 pp	NIE
dili	0.8418	0.7770	0.8302	VIRTUAL_NODE	0.8657	+3.55 pp	+8.87 pp	TAK	+2.39 pp	TAK
skin_reaction	0.7270	0.6741	0.6212	UNCERTAINTY	0.7455	+12.44 pp	+7.14 pp	TAK	+1.85 pp	TAK
ames	0.8735	0.8725	0.8764	EDGE_FEAT	0.8872	+1.08 pp	+1.47 pp	TAK	+1.37 pp	TAK
clintox	0.8804	0.7700	0.8048	PCGRAD	0.8382	+3.33 pp	+6.82 pp	TAK	-4.23 pp	NIE

Wariant: GNN_RDKIT | Bije STL: 13/13 | Bije XGB: 10/13

Task	XGBoost	STL_GNN	MAIN	Najlepsza metoda	Najlepszy AUROC	Delta vs MAIN [pp]	Delta vs STL [pp]	MTL bije STL?	Delta vs XGB [pp]	MTL bije XGB?
hia_hou	0.9875	0.9826	0.9691	EDGE_FEAT	0.9940	+2.50 pp	+1.14 pp	TAK	+0.65 pp	TAK
pgp_broccatelli	0.9328	0.8922	0.9208	EDGE_FEAT	0.9315	+1.06 pp	+3.93 pp	TAK	-0.13 pp	NIE
bbb_martins	0.8943	0.8296	0.8801	VIRTUAL_NODE	0.9187	+3.86 pp	+8.90 pp	TAK	+2.44 pp	TAK
cyp2c19_veith	0.8891	0.8870	0.8839	GROUPING	0.9019	+1.79 pp	+1.49 pp	TAK	+1.28 pp	TAK
cyp2d6_veith	0.8580	0.8424	0.8408	PCGRAD	0.8644	+2.37 pp	+2.20 pp	TAK	+0.64 pp	TAK
cyp3a4_veith	0.8792	0.8881	0.8733	GROUPING	0.9008	+2.74 pp	+1.26 pp	TAK	+2.15 pp	TAK
cyp1a2_veith	0.9312	0.9259	0.9121	EDGE_FEAT	0.9374	+2.53 pp	+1.15 pp	TAK	+0.63 pp	TAK
cyp2c9_veith	0.8878	0.8748	0.8879	GROUPING	0.9060	+1.81 pp	+3.12 pp	TAK	+1.82 pp	TAK
herg	0.8211	0.8372	0.8023	UNCERTAINTY	0.8839	+8.16 pp	+4.67 pp	TAK	+6.28 pp	TAK
dili	0.8803	0.7921	0.8125	VIRTUAL_NODE	0.9207	+10.82 pp	+12.85 pp	TAK	+4.03 pp	TAK
skin_reaction	0.7883	0.7009	0.6824	GROUPING	0.7443	+6.19 pp	+4.34 pp	TAK	-4.40 pp	NIE
ames	0.8964	0.8597	0.8467	GROUPING	0.8883	+4.16 pp	+2.86 pp	TAK	-0.81 pp	NIE
clintox	0.9140	0.9104	0.8880	PCGRAD	0.9422	+5.42 pp	+3.18 pp	TAK	+2.82 pp	TAK

Wariant: GNN_RDKIT_MORGAN | Bije STL: 13/13 | Bije XGB: 11/13

Task	XGBoost	STL_GNN	MAIN	Najlepsza metoda	Najlepszy AUROC	Delta vs MAIN [pp]	Delta vs STL [pp]	MTL bije STL?	Delta vs XGB [pp]	MTL bije XGB?
hia_hou	0.9864	0.9826	0.9810	VIRTUAL_NODE	0.9957	+1.47 pp	+1.30 pp	TAK	+0.92 pp	TAK
pgp_broccatelli	0.9342	0.9046	0.9344	GROUPING	0.9356	+0.12 pp	+3.10 pp	TAK	+0.14 pp	TAK
bbb_martins	0.9008	0.8650	0.8769	PCGRAD	0.8861	+0.92 pp	+2.12 pp	TAK	-1.46 pp	NIE
cyp2c19_veith	0.8913	0.8846	0.9073	MAIN	0.9073	+0.00 pp	+2.27 pp	TAK	+1.60 pp	TAK
cyp2d6_veith	0.8654	0.8403	0.8691	MAIN	0.8691	+0.00 pp	+2.88 pp	TAK	+0.37 pp	TAK
cyp3a4_veith	0.8889	0.8892	0.8984	GROUPING	0.9036	+0.52 pp	+1.44 pp	TAK	+1.47 pp	TAK
cyp1a2_veith	0.9345	0.9261	0.9386	GROUPING	0.9388	+0.02 pp	+1.27 pp	TAK	+0.44 pp	TAK
cyp2c9_veith	0.8986	0.8770	0.9071	GROUPING	0.9113	+0.41 pp	+3.42 pp	TAK	+1.27 pp	TAK
herg	0.8162	0.8069	0.8186	CROSS_UNCERT	0.8653	+4.67 pp	+5.84 pp	TAK	+4.92 pp	TAK
dili	0.8648	0.8218	0.8457	VIRTUAL_NODE	0.8918	+4.61 pp	+7.00 pp	TAK	+2.70 pp	TAK
skin_reaction	0.7551	0.6346	0.7124	BALANCING	0.7812	+6.89 pp	+14.67 pp	TAK	+2.61 pp	TAK
ames	0.8930	0.8570	0.8789	PCGRAD	0.8812	+0.23 pp	+2.42 pp	TAK	-1.19 pp	NIE
clintox	0.9226	0.9066	0.9040	PCGRAD	0.9478	+4.39 pp	+4.12 pp	TAK	+2.53 pp	TAK

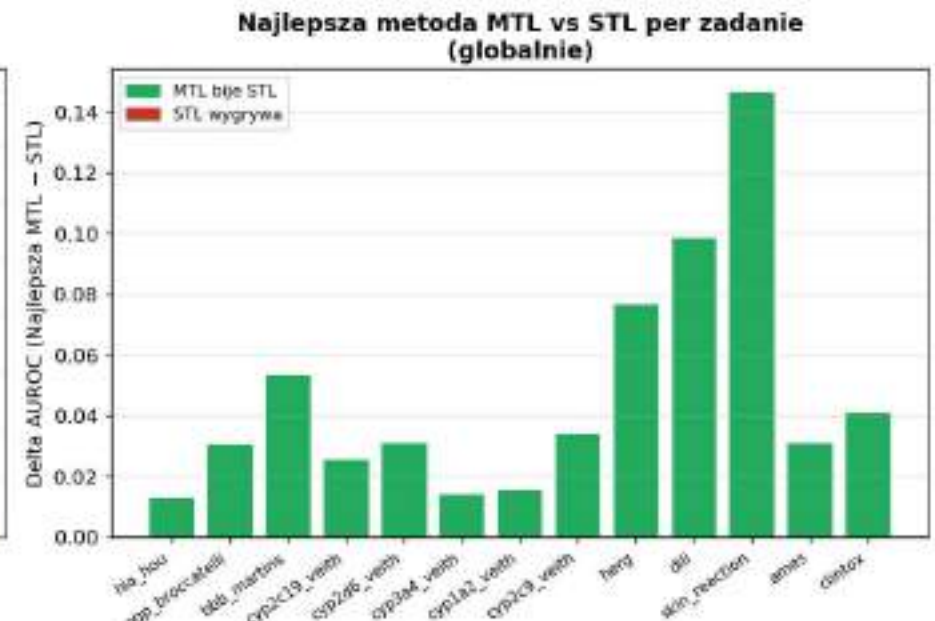
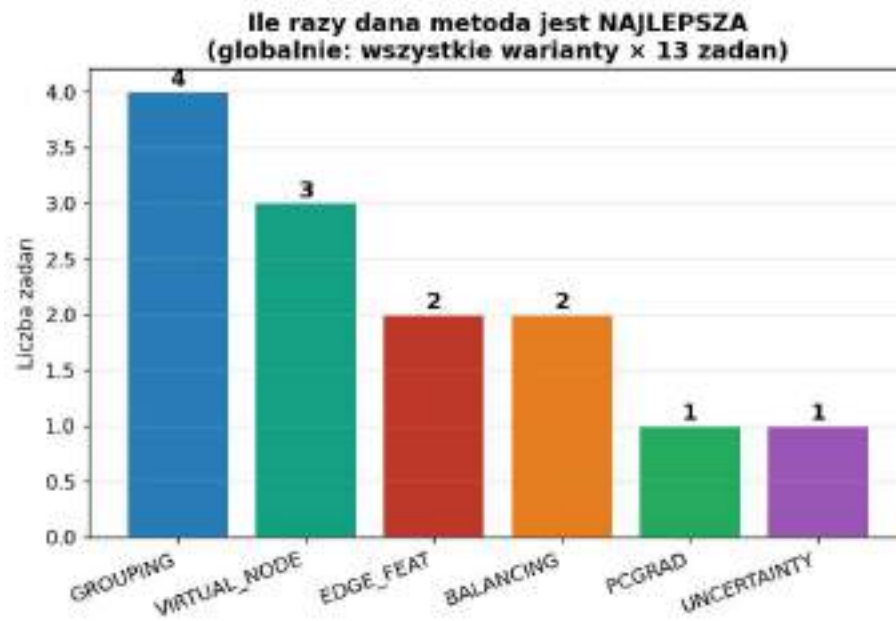
```
In [9]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Lewy: ile razy najlepsza (globalnie)
ax = axes[0]
wc = global_best['Metoda'].value_counts()
colors_w = [METHOD_COLORS.get(m, '#7f8c8d') for m in wc.index]
bars = ax.bar(wc.index, wc.values, color=colors_w, edgecolor='white')
for bar, val in zip(bars, wc.values):
    ax.text(bar.get_x() + bar.get_width()/2, val + 0.05, str(val),
            ha='center', fontsize=12, fontweight='bold')
```

```
ax.set_title('Ile razy dana metoda jest NAJLEPSZA\n(globalnie: wszystkie warianty x 13 zadan)', fontweight='bold')
ax.set_ylabel('Liczba zadan')
ax.set_xticklabels(wc.index, rotation=20, ha='right')
ax.grid(axis='y', alpha=0.3)

# Prawy: best MTL vs STL per task
ax2 = axes[1]
gb_auroc = global_best['AUROC'].reindex(stl_ref.index)
best_vs_stl = gb_auroc - stl_ref
bc = ['#27ae60' if v > 0 else '#c0392b' for v in best_vs_stl]
ax2.bar(best_vs_stl.index, best_vs_stl.values, color=bc, edgecolor='white')
ax2.axhline(0, color='black', lw=1.2)
ax2.set_xticklabels(best_vs_stl.index, rotation=35, ha='right', fontsize=8.5)
ax2.set_ylabel('Delta AUROC (Najlepsza MTL - STL)')
ax2.set_title('Najlepsza metoda MTL vs STL per zadanie\n(globalnie)', fontweight='bold')
ax2.grid(axis='y', alpha=0.3)
ax2.legend(handles=[
    mpatches.Patch(color='#27ae60', label='MTL bije STL'),
    mpatches.Patch(color='#c0392b', label='STL wygrywa'),
], fontsize=9)

plt.tight_layout()
plt.show()
```



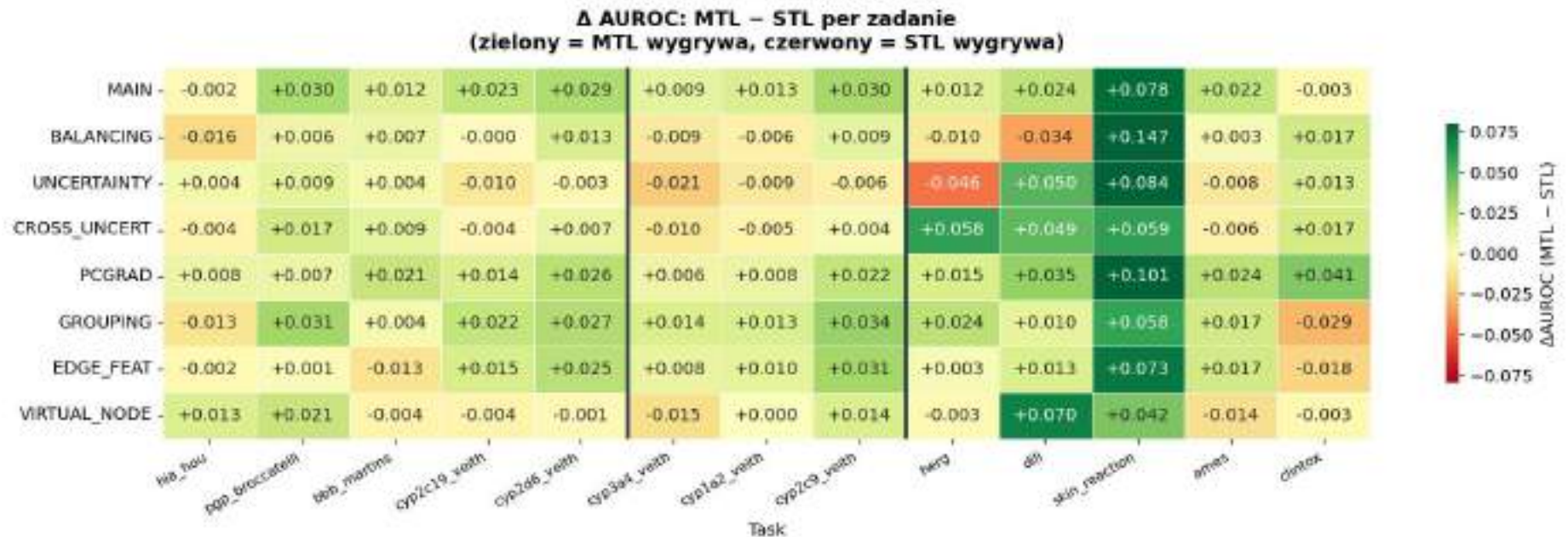
6. Heatmapa Delta: MTL – STL per metoda

```
In [18]: mtl_methods = [c for c in full.columns if c != 'STL_GNN']
vs_stl = full[mtl_methods].subtract(full['STL_GNN'], axis=0)

fig, ax = plt.subplots(figsize=(15, 5))
sns.heatmap(vs_stl.T.astype(float), annot=True, fmt='+.3f',
            cmap='RdYlGn', center=0, linewidths=0.4, linecolor='white',
            vmin=-0.08, vmax=0.08, ax=ax,
            cbar_kws={'label': 'ΔAUROC (MTL - STL)', 'shrink': 0.7})
ax.set_title('Δ AUROC: MTL - STL per zadanie\n(zielony = MTL wygrywa, czerwony = STL wygrywa)',
            fontweight='bold', pad=12)
ax.set_xticklabels(ax.get_xticklabels(), rotation=30, ha='right', fontsize=9)
for b in [5, 8]:
    ax.axvline(b, color='#2c3e50', linewidth=2.5)
plt.tight_layout()
plt.show()

beats_stl = (vs_stl > 0).sum()
```

```
display(pd.DataFrame({'Bije STL z 13': beats_stl, 'Sr. Δ vs STL [pp]': (vs_stl.mean()*100).round(2)})
        .sort_values('Bije STL z 13', ascending=False)
        .style.format({'Sr. Δ vs STL [pp]': '{:+.2f} pp'})
        .applymap(lambda v: 'background-color: #27ae6033; font-weight: bold' if isinstance(v,(int,float)) and v>=10
                    else ('background-color: #f39c1233' if isinstance(v,(int,float)) and v>=7
                        else ('background-color: #c0392b22' if isinstance(v,(int,float)) else '')),
        subset=['Bije STL z 13'])
        .set_caption('Ile zadań każda metoda MTL bije STL'))
```



Ile zadaní každá metoda MTL bije STL

	Bije STL z 13	Sr. Δ vs STL [pp]
PCGRAD	13	+2.52 pp
MAIN	11	+2.12 pp
GROUPING	11	+1.64 pp
EDGE_FEAT	10	+1.26 pp
CROSS_UNCERT	8	+1.47 pp
BALANCING	7	+0.99 pp
UNCERTAINTY	6	+0.47 pp
VIRTUAL_NODE	6	+0.90 pp

7. Analiza per klaster zadaní

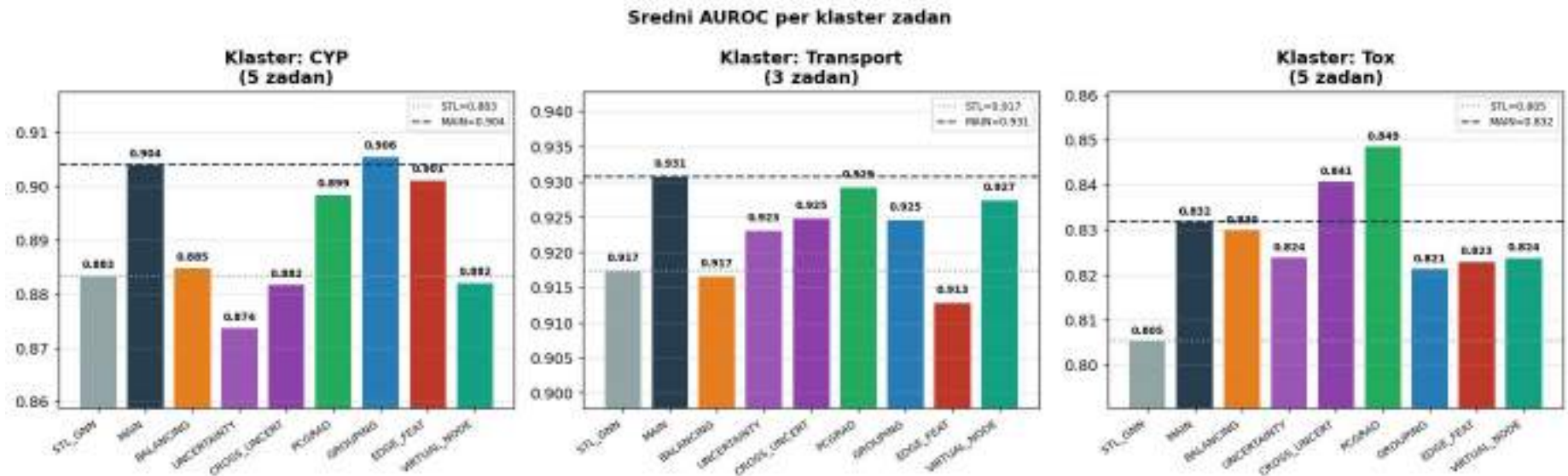
```
In [11]: fig, axes = plt.subplots(1, 3, figsize=(16, 5))
all_cols = ['STL_GNN'] + [c for c in full.columns if c != 'STL_GNN']

for ax, (cluster, tasks) in zip(axes, TASK_CLUSTERS.items()):
    ct = [t for t in tasks if t in full.index]
    means_c = full.loc[ct, all_cols].mean()
    colors_c = [METHOD_COLORS.get(m, '#7f8c8d') for m in means_c.index]
    bars = ax.bar(means_c.index, means_c.values, color=colors_c, edgecolor='white')
    ax.axhline(means_c['STL_GNN'], color='#95a5a6', linestyle=':', lw=1.5,
               label=f'STL={means_c["STL_GNN"]:.3f}')
    ax.axhline(means_c['MAIN'], color='#2c3e50', linestyle='--', lw=1.5,
               label=f'MAIN={means_c["MAIN"]:.3f}')

    for bar, val in zip(bars, means_c.values):
        ax.text(bar.get_x() + bar.get_width()/2, val + 0.001, f'{val:.3f}',
                ha='center', va='bottom', fontsize=7.5, fontweight='bold')
    ax.set_title(f'Klaster: {cluster}\n({len(ct)} zadaní)', fontweight='bold')
    ax.set_ylim(min(means_c.values) - 0.015, max(means_c.values) + 0.012)
    ax.set_xticklabels(means_c.index, rotation=35, ha='right', fontsize=8)
```

```
ax.legend(fontsize=7.5)
ax.grid(axis='y', alpha=0.3)
```

```
plt.suptitle('Sredni AUROC per klaster zadan', fontweight='bold', fontsize=13)
plt.tight_layout()
plt.show()
```



8. Zadania problemowe: herg, dili, skin_reaction

```
In [12]: problem_tasks = [t for t in ['herg', 'dili', 'skin_reaction'] if t in full.index]
all_cols = ['STL_GNN'] + [c for c in full.columns if c != 'STL_GNN']

fig, axes = plt.subplots(1, len(problem_tasks), figsize=(14, 5))

for ax, task in zip(axes, problem_tasks):
    vals = full.loc[task, all_cols]
    bcolors = [METHOD_COLORS.get(m, '#7f8c8d') for m in vals.index]
    bars = ax.bar(vals.index, vals.values, color=bcolors, edgecolor='white')
    stl_v = vals['STL_GNN']
    ax.axhline(stl_v, color='#95a5a6', linestyle=':', lw=1.8, label=f'STL={stl_v:.3f}')

    for bar, val in zip(bars, vals.values):
```

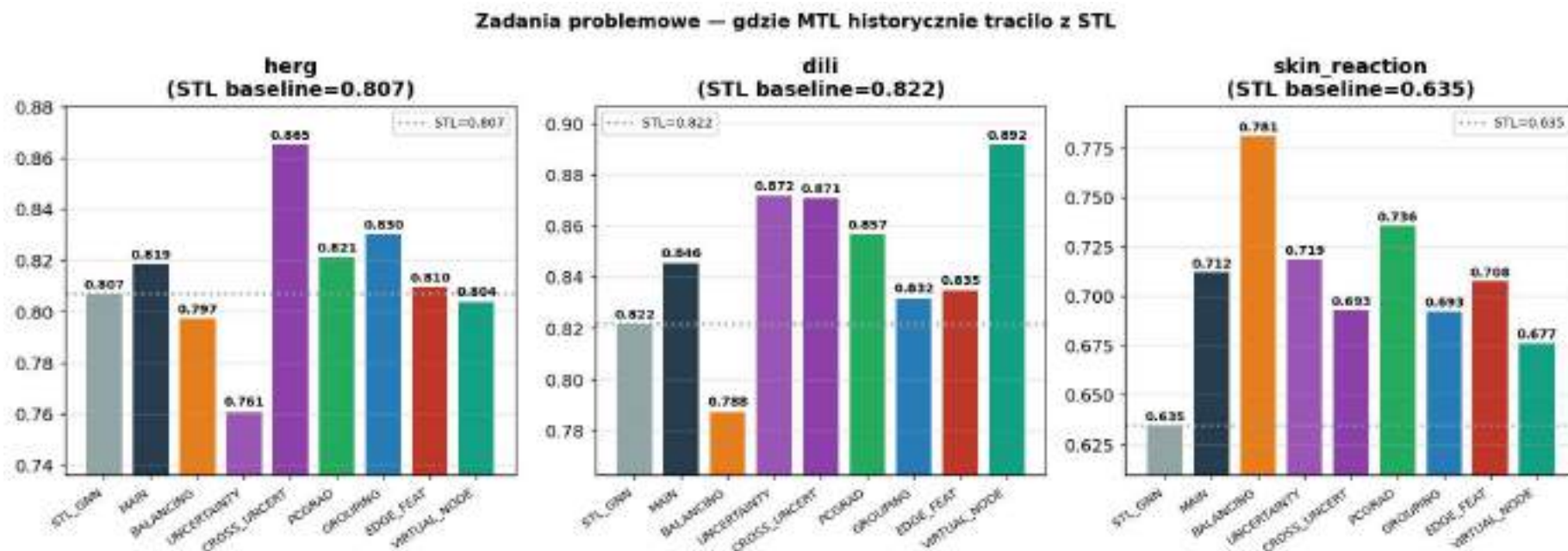


```

ax.text(bar.get_x() + bar.get_width()/2, val + 0.001, f'{val:.3f}',
        ha='center', va='bottom', fontsize=8, fontweight='bold')
ax.set_title(f'{task}\n(STL baseline={stl_v:.3f})', fontweight='bold')
ax.set_ylim(min(vals.values) - 0.025, max(vals.values) + 0.015)
ax.set_xticklabels(vals.index, rotation=35, ha='right', fontsize=8)
ax.legend(fontsize=8)
ax.grid(axis='y', alpha=0.3)

plt.suptitle('Zadania problemowe – gdzie MTL historycznie traciło z STL',
            fontweight='bold', fontsize=12)
plt.tight_layout()
plt.show()

```



9. Wpływ wariantu wejść: GNN vs +Morgan vs +RDKit vs +oba

```

In [13]: VARIANTS = ['GNN', 'GNN_MORGAN', 'GNN_RDKIT', 'GNN_RDKIT_MORGAN']
VARIANT_COLORS = {'GNN': '#3498db', 'GNN_MORGAN': '#e67e22',
                  'GNN_RDKIT': '#9b59b6', 'GNN_RDKIT_MORGAN': '#27ae60'}

```

```

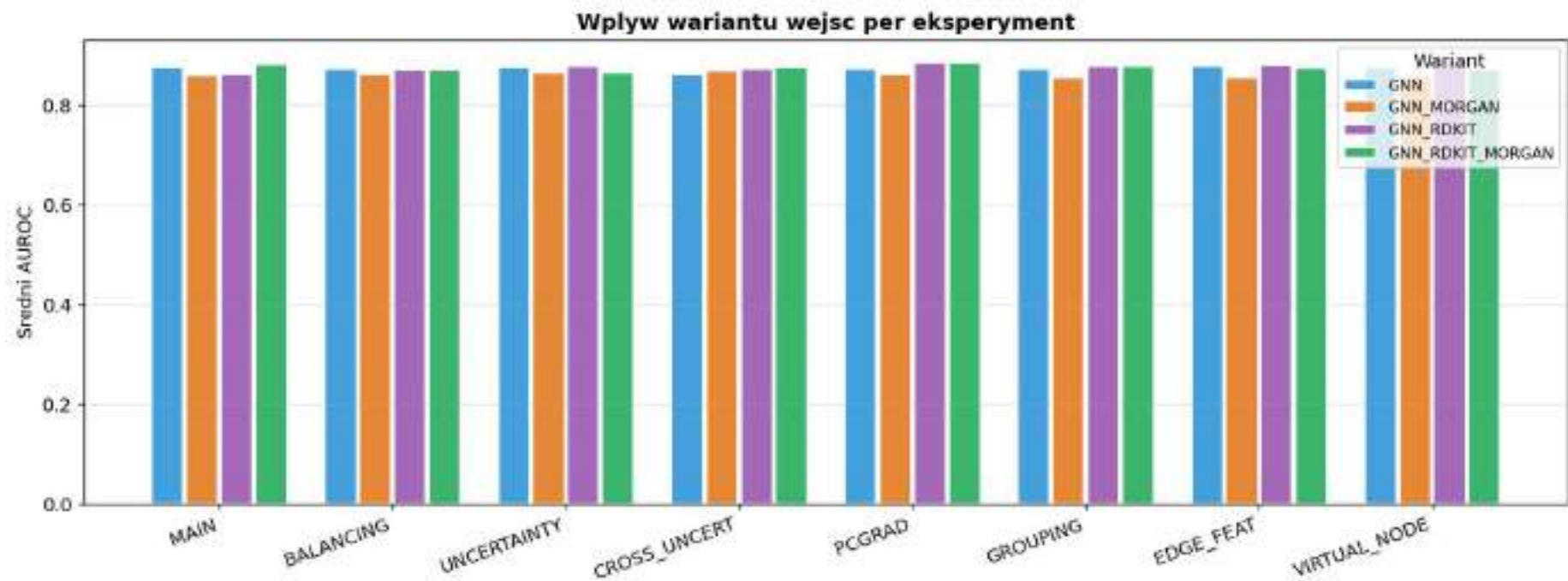
variant_means = {}
for exp_label, (exp_dir, col) in EXPERIMENTS.items():
    row = {}
    for var in VARIANTS:
        p = os.path.join(BASE, exp_dir, var, 'final_results.csv')
        if os.path.exists(p):
            df = pd.read_csv(p).set_index('Task')
            if col in df.columns:
                row[var] = df[col].mean()
    if row:
        variant_means[exp_label] = row

vdf = pd.DataFrame(variant_means).T

fig, ax = plt.subplots(figsize=(13, 5))
x = np.arange(len(vdf.index))
width = 0.2
offsets = np.array([-1.5, -0.5, 0.5, 1.5]) * width
for i, var in enumerate(VARIANTS):
    if var in vdf.columns:
        ax.bar(x + offsets[i], vdf[var], width=width*0.9,
              color=VARIANT_COLORS[var], label=var, edgecolor='white', alpha=0.9)
ax.set_xticks(x)
ax.set_xticklabels(vdf.index, rotation=20, ha='right')
ax.set_ylabel('Sredni AUROC')
ax.set_title('Wplyw wariantu wejsc per eksperyment', fontweight='bold')
ax.legend(title='Wariant', fontsize=9)
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

print('Sredni AUROC per eksperyment x wariant wejsc:')
display(vdf.round(4).style.background_gradient(cmap='YlGn', axis=None))

```



Sredni AUROC per eksperyment x wariant wejsc:

	GNN	GNN_MORGAN	GNN_RDKIT	GNN_RDKIT_MORGAN
MAIN	0.876400	0.859000	0.861500	0.882500
BALANCING	0.872900	0.860500	0.871600	0.871100
UNCERTAINTY	0.876500	0.866100	0.878700	0.866000
CROSS_UNCERT	0.861800	0.868000	0.872200	0.876000
PCGRAD	0.873100	0.862200	0.886500	0.886400
GROUPING	0.873400	0.855800	0.878500	0.877600
EDGE_FEAT	0.879200	0.856200	0.879800	0.873800
VIRTUAL_NODE	0.877000	0.857400	0.887400	0.870200

10. Wnioski końcowe

```

In [14]: print('=' * 68)
print(' PODSUMOWANIE EKSPERYMENTOW MTL ADMET')
print('=' * 68)
print(f' Wariant:          {VARIANT}')
print(f' Zadania:          {len(TASKS)}')
print(f' STL baseline:      {full["STL_GNN"].mean():.4f}')
main_m = full['MAIN'].mean()
stl_m = full['STL_GNN'].mean()
print(f' MAIN (MTL base):   {main_m:.4f} ({main_m - stl_m:+.4f} vs STL)')
print(f' MAIN bije STL na:  {(full["MAIN"] > full["STL_GNN"]).sum()}/13 zadaniach')
if xgb is not None:
    print(f' XGBoost baseline: {xgb.mean():.4f} (per-task – patrz tabele w sek. 5)')
    print(f' MAIN bije XGB na: {(full["MAIN"] > xgb).sum()}/13 zadaniach')
print()

best_overall = full.mean().idxmax()
print(f' NAJLEPSZA METODA GLOBALNIE: {best_overall} ({full.mean().max():.4f}')
print()
print(' Najlepsza metoda per zadanie:')
bm = full.idxmax(axis=1)
bs = full.max(axis=1)
for task in TASKS:
    flag = 'TAK' if bs[task] > stl[task] else 'NIE'
    cluster = next((c for c, ts in TASK_CLUSTERS.items() if task in ts), '?')
    print(f' [{flag}] [{cluster:<10}] {task:<30} -> {bm[task]:<15} ({bs[task]:.4f}')

print()
print(' Metody po liczbie zwyciestwach w per-task:')
for m, cnt in bm.value_counts().items():
    print(f' {m:<15} {cnt:>2}/13 zadan')
print()
print(' MTL vs STL – ile zadan wygrywa kazda metoda:')
for m in [c for c in full.columns if c != 'STL_GNN']:
    cnt = (full[m] > full['STL_GNN']).sum()
    avg = (full[m] - full['STL_GNN']).mean() * 100
    print(f' {m:<15} {cnt:>2}/13 avg_delta={avg:+.2f} pp')
print('=' * 68)

```

```
=====
PODSUMOWANIE EKSPERYMENTOW MTL ADMET
=====
```

```
Wariant:      GNN_RDKIT_MORGAN
Zadania:      13
STL baseline: 0.8612
MAIN (MTL base): 0.8825 (+0.0212 vs STL)
MAIN bije STL na: 11/13 zadaniach
XGBoost baseline: 0.8886 (per-task - patrz tabele w sek. 5)
MAIN bije XGB na: 7/13 zadaniach
```

```
NAJLEPSZA METODA GLOBALNIE: PCGRAD (0.8864)
```

```
Najlepsza metoda per zadanie:
```

[TAK] [Transport] h1a_hou	-> VIRTUAL_NODE	(0.9957)
[TAK] [Transport] pgp_broccatelli	-> GROUPING	(0.9356)
[TAK] [Transport] bbb_martins	-> PCGRAD	(0.8861)
[TAK] [CYP] cyp2c19_veith	-> MAIN	(0.9073)
[TAK] [CYP] cyp2d6_veith	-> MAIN	(0.8691)
[TAK] [CYP] cyp3a4_veith	-> GROUPING	(0.9036)
[TAK] [CYP] cyp1a2_veith	-> GROUPING	(0.9388)
[TAK] [CYP] cyp2c9_veith	-> GROUPING	(0.9113)
[TAK] [Tox] herg	-> CROSS_UNCERT	(0.8653)
[TAK] [Tox] dili	-> VIRTUAL_NODE	(0.8918)
[TAK] [Tox] skin_reaction	-> BALANCING	(0.7812)
[TAK] [Tox] ames	-> PCGRAD	(0.8812)
[TAK] [Tox] clintox	-> PCGRAD	(0.9478)

```
Metody po liczbie zwyciestwach w per-task:
```

GROUPING	4/13 zadan
PCGRAD	3/13 zadan
VIRTUAL_NODE	2/13 zadan
MAIN	2/13 zadan
CROSS_UNCERT	1/13 zadan
BALANCING	1/13 zadan

```
MTL vs STL - ile zadan wygrywa kazda metoda:
```

MAIN	11/13	avg_delta=+2.12 pp
BALANCING	7/13	avg_delta=+0.99 pp
UNCERTAINTY	6/13	avg_delta=+0.47 pp
CROSS_UNCERT	8/13	avg_delta=+1.47 pp

PCGRAD	13/13	avg_delta=+2.52 pp
GROUPING	11/13	avg_delta=+1.64 pp
EDGE_FEAT	18/13	avg_delta=+1.26 pp
VIRTUAL_NODE	6/13	avg_delta=+0.98 pp

=====

11. Szczegółowy opis metod — motywacja chemiczna i techniczna

BALANCING — Ważenie klas (Class-Balanced Loss)

Typ: Modyfikacja funkcji kosztu

Zmiana: `BCEWithLogitsLoss(pos_weight = n_neg / n_pos)` — obliczane osobno per task ze zbioru treningowego.

Kontekst chemiczny i medyczny: Dane ADMET są z natury mocno niezbilansowane. Wynika to z biochemicznej rzeczywistości: większość cząsteczek *nie* hamuje CYP, *nie* wywołuje hepatotoksyczności, *nie* przenika BBB. W zbiorze `skin_reaction` (~15% pozytywnych) standardowy BCE osiąga wysoką dokładność, klasyfikując wszystko jako "bezpieczne" — i to wystarcza, żeby loss był mały. Model ignoruje rzadkie przypadki toksyczności, bo za bardzo nie "boli" go pomyłka na mniejszościowej klasie.

W praktyce farmaceutycznej jest to krytyczny błąd: pominięcie toksycznej cząsteczki jest gorsze niż odrzucenie bezpiecznej. Ważenie klas wymusza symetrię — każdy fałszywy negatyw na klasie pozytywnej kosztuje tyle samo co fałszywy pozytyw.

Dlaczego nie działa globalnie: Ważenie klas zmienia efektywny gradient per task. Taski, które mają już dobrą równowagę (CYP: ~30-40% pozytywnych), dostają destabilizację — model "przekalibrowany" na rzadkie zdarzenia zaczyna agresywnie przewidywać pozytywne wyniki tam, gdzie ich nie ma. Efekt: duży zysk na `skin_reaction` (+6.9 pp), ale degradacja na CYP.

UNCERTAINTY — Homoscedastic Uncertainty Weighting (Kendall & Gal, NeurIPS 2017)

Typ: Modyfikacja funkcji kosztu

Zmiana: Każdy task dostaje uczony parametr `log σ²_t`. Łączna strata: $L = \sum_t [L_t / \sigma^2_t + \log \sigma_t]$

Kontekst: Hipoteza Kendall & Gal: nie wszystkie zadania są równie "pewne" — `skin_reaction` ma dużo szumu i małą spójność strukturę-aktywność (SAR), `cyp2c19` jest dobrze zbadany i łatwy do modelowania. Zamiast traktować oba tak samo, niech model sam nauczy się, ile wagi dać każdemu taskowi.

Mechanizm:

- Duże σ^2_t → task jest "trudny/niepewny" → jego gradient ma małą wagę → mniej zaburza wspólny backbone
- Regularyzacja $+\log \sigma_t$ zapobiega wyciszeniu wszystkich tasków do $\sigma \rightarrow \infty$

Dlaczego nie działa dobrze na ADMET: Model nauczył się wyciszać *trudne* taski (duże σ^2), a nie *kolidujące*. Tymczasem trudność i konfliktowość to dwie różne rzeczy. `heng` i `d111` mają inne mechanizmy niż CYP, a więc ich gradienty *kolidują* z CYP-owymi, ale każdy z nich oddzielnie jest "pewny" (duże, czyste datasety). Uncertainty weighting źle diagnozuje problem.

CROSS_UNCERTAINTY — Macierz Cross-Task Uncertainty

Typ: Modyfikacja funkcji kosztu

Zmiana: Macierz $\sigma[i, j]$ 13×13 . Task `i` może wyciszyć wkład taska `j` do swojej straty: $L_i = \sum_j [L_j / \sigma[i, j]^2 + \log \sigma[i, j]]$

Motywacja biochemiczna: Taski ADMET nie są równorzędne. CYP3A4 odpowiada za metabolizm ~50% leków na rynku — jego sygnał jest silny i rzetelny, hERG (kardiotoksyczność — blokowanie kanału potasowego serca) ma zupełnie inny mechanizm i inną SAR. Hipoteza: niech model CYP sam zdecyduje, ile "słucha" gradienta z hERG.

Co model rzeczywiście się nauczył: Off-diagonal $\sigma[i, j]$ wzrosły dla większości par — model zaczął wyciszać *wzajemnie* prawie wszystko. Zostaje kilka silnych par (np. CYP2C9 vs CYP3A4 — oba metabolizują podobne substraty) plus `HERG` wyciszony przez CYP. Stąd spektakularny zysk na hERG (+4.67 pp) — usunięcie zakłócenia z CYP uwolniło sygnał z danych kardiotoksycznych — ale globalna degradacja, bo reszta tasków też "odcięła się" od informacji wspólnej.

PCGRAD — Gradient Surgery (Yu et al., NeurIPS 2020)

Typ: Modyfikacja procedury optymalizacji

Zmiana: Dla każdej pary tasków (i, j) : jeśli $\text{dot}(\nabla_i, \nabla_j) < 0$, projekcja $\nabla_i + \nabla_j - (\nabla_i \cdot \nabla_j / ||\nabla_j||^2) \nabla_j$

Dlaczego gradienty kolidują: Wyobraź sobie gradient dla CYP2C9 jako wektor w przestrzeni parametrów: "zwiększ wagę reprezentacji aromatycznych pierścieni metabolizowanych przez CYP". Gradient dla hERG może wskazywać w przeciwnym kierunku: "zmniejsz wrażliwość na aromatyczne pierścienie — hERG-blockers to często płaskie aromatyki, których model NIE chce identyfikować jako metaboliczne substraty". Iloczyn skalarny jest ujemny → gradienty aktywnie sobie szkodzą → krok optymalizatora idzie w złym kierunku dla obu.

Działanie PCGrad: Zamiast sumować kolidujące gradienty, projekt gradientu taska i na podprzestrzeń prostopadłą do gradientu taska j . Każdy task "naprawia" swoje parametry w kierunku, który nie szkodzi innemu. Backbone wspólny staje się kompatybilnym kompromisem zamiast polem bitwy.

Wynik i interpretacja: PCGrad to **jedyna metoda bijąca MAIN globalnie** (+2.52 pp vs STL, 13/13 tasków > STL). Działa bez zmian architektury i danych. Potwierdza hipotezę: głównym problemem MTL na ADMET są *konflikty gradientów*, nie niedopasowanie danych ani architektura.

GROUPING — Dual-Backbone Architecture (CYP vs reszta)

Typ: Modyfikacja architektury

Zmiana: Dwa backbone GIN: `gin_cyp` (5 tasków CYP) + `gin_other` (8 tasków). Wspólny wektor cech cząsteczkowych (RDKit/Morgan). Osobne głowice na każdy task.

Uzasadnienie biochemiczne: Enzymy CYP (cytochrom P450) to oksydaza monooksygenazowa — katalizuje utlenianie ksenobiotyków w wątrobie. Pięć izoform CYP w datasetcie (CYP1A2, CYP2C9, CYP2C19, CYP2D6, CYP3A4) ma podobne mechanizmy wiązania substratu: hydrofobowa kieszeń aktywna, specyficzność steryczna, podobne cechy strukturalne predyktywne (aromatyczność, lipofilność, obecność grup azotowych). Ich SAR jest wzajemnie zbliżona — mogą dzielić backbone.

Z kolei hERG to kanał jonowy serca (K^+). BBB zależy od MW, logP, TPSA. DILI to hepatotoksyczność przez reaktywne metabolity. Mechanizmy są fundamentalnie różne od metabolizmu CYP — gradient z CYP może *aktywnie przeszkadzać* w uczeniu właściwości globalnych.

Dlaczego nie pomaga w pełni: Izolacja CYP ≠ izolacja konfliktów dla tasków spoza CYP. `herg` , `dili` , `skin_reaction` nadal dzielą `gin_other` backbone i nadal kolidują. Grouping naprawia połowę problemu (CYP-część), ale nie resztę.

EDGE_FEATURES — GINEConv z cechami wiązań (Hu et al., ICLR 2020)

Typ: Modyfikacja architektury sieci GNN

Zmiana: GINConv $\rightarrow$ GINEConv + 6-wymiarowy wektor `edge_attr` per wiązanie:

- Typ wiązania: SINGLE / DOUBLE / TRIPLE / AROMATIC (4 bity one-hot)
- `IsInRing` — czy wiązanie należy do pierścienia
- `IsConjugated` — czy wiązanie jest skoniugowane

Dlaczego informacja o wiązaniach jest chemicznie kluczowa: Standardowy GINConv traktuje każde wiązanie identycznie — to tak jakby reprezentować cząsteczkę tylko listą atomów sąsiadujących, bez informacji o naturze połączenia. Tymczasem w kontekście ADMET:

- **Metabolizm CYP:** Enzymy CYP atakują głównie sp^3 -węgle (C-H utlenianie) i konkretne grupy funkcyjne. Wiązanie C=C (podwójne) lub C $\equiv$ N (potrójne) jest metabolizowane inaczej niż C-C. Aromatyczne pierścienie mogą przechodzić epoksydację. GINConv tego nie widzi.
- **BBB i lipofilność:** Układy sprzężone (wiązania skoniugowane) podwyższają sztywność i planarność cząsteczki, co wpływa na jej transport przez błony lipidowe. Pierścieniowe i aromatyczne fragmenty dominują w cząsteczkach "CNS-like" przekraczających BBB.
- **hERG:** Kanał hERG blokują głównie płaskie, aromatyczne cząsteczki z podstawnikami aminowymi — struktura aromatu (krawędzie AROMATIC) powinna być kluczowym sygnałem.
- **DILI:** Hepatotoksyczne "ostrzegawcze grupy" (structural alerts) jak nitro, anilina, aldehyd — często identyfikowane przez typ wiązania i kontekst sąsiedni.

Dlaczego wynik jest słaby: GINEConv przekazuje informację o krawędziach przez $h_v + \text{MLP}(h_v + e_{\{vw\}})$ dla każdego sąsiada. Przy 3 warstwach model *widzi* krawędzie, ale 3-hop zasięg może być za mały, żeby zintegrować globalny kontekst wiązań (np. cały układ skoniugowany cząsteczki). Ponadto 6-bitowe one-hot `edge_attr` są proste — brakuje cech jak energia wiązania, długość, kąty torsyjne. Wynik sugeruje, że *topologia* (kto jest sąsiadem kogo) w 3-hop GIN już niesie większość sygnału dostępnego z tej prostej reprezentacji krawędzi.

VIRTUAL_NODE — Globalny węzeł komunikacyjny

Typ: Modyfikacja preprocessingu danych (augmentacja grafu)

Zmiana: Do każdej cząsteczki dodawany węzeł VN z zerowym wektorem cech $[0, \dots, 0]$ (9-wymiarowym), połączony ze *wszystkimi* atomami bidirekcyjnie.

Problem, który rozwiązuje — fizyczne uzasadnienie: Standardowy 3-warstwowy GIN widzi tylko **3-hop neighborhood** każdego atomu. Dla dużej cząsteczki (~40 atomów) atom na jednym końcu łańcucha nigdy nie "widzi" atomu na przeciwnym końcu. Tymczasem wiele właściwości ADMET to właściwości *globalne*:

- **Masa cząsteczkowa (MW):** decyduje o filtracji przez BBB i nerkach — wynika z *sumy* wszystkich atomów
- **logP (lipofilność):** wynika z bilansu grup hydrofilowych i hydrofobowych w całej cząsteczce — lokalna informacja 3-hop jest niewystarczająca
- **TPSA (topological polar surface area):** suma wszystkich biegunowych grup — właściwość globalna wpływająca na wchłanianie
- **Reaktywność metaboliczna (DILI):** cząsteczki tworzące reaktywne metabolity (chinony, epoksydy) mają specyficzne fragmenty w *kontekście całej struktury*, nie tylko lokalnie

Mechanizm VN: Po pierwszej warstwie GIN węzeł VN agreguje informacje ze *wszystkich* atomów (suma pool): $h_{VN} = AGG(\{h_v : v \in V\})$. W drugiej warstwie każdy atom dostaje z powrotem h_{VN} jako "globalny kontekst". Efektywnie: 2 warstwy z VN $\approx$ nieskończony zasięg.

Wyniki i interpretacja:

- **dili** +4.61 pp — hepatotoksyczność (DILI) jest jednym z najtrudniejszych endpointów ADMET, zależy od globalnej reaktywności metabolicznej. VN umożliwia modelowi widzieć całą cząsteczkę jednocześnie ✓
- **hia_hou** (wchłanianie jelitowe) +1.47 pp — wchłanianie zależy od MW, lipofilności, TPSA — właściwości globalne ✓
- CYP degradacja: enzymy CYP rozpoznają lokalne miejsca aktywne (konkretna grupa funkcyjna w kieszeni enzymowej). VN "miesza" informację globalną do lokalnej reprezentacji atomów, zaburzając lokalny sygnał wystarczający dla CYP.

12. Wnioski techniczne i biochemiczne

12.1 Główny bottleneck MTL na ADMET: konflikty gradientów

Eksperymenty pokazują, że standardowe MTL na zadaniach ADMET *nie jest* ograniczone brakiem danych ani słabą architekturą — ograniczone jest **konfliktami gradientów** między mechanistycznie różnymi endpointami.

Potwierdzenie:

- **PCGrad** (Gradient Surgery) bije wszystkie inne metody globalnie (+2.52 pp vs STL) **bez zmian architektury ani danych** — po samym naprawieniu kolizji gradientów

- **CROSS_UNCERTAINTY** wyciszając cross-task zyska na hERG +4.67 pp, co dowodzi, że gradient CYP aktywnie szkodził modelowi hERG
- **GROUPING** poprawia CYP (+0.4 pp) przez izolację backboneu, ale nie naprawia hERG/DILI — co sugeruje, że problem dotyczy *konkretnych par* (CYP→hERG), nie całych grup

12.2 Trzy "trudne zadania" — dlaczego MTL historycznie tu przegrywa

Zadanie	Mechanizm biologiczny	Dlaczego MTL szkodzi
hERG	Blokowanie kanału K ⁺ serca (IKr). Kardiotoksyczność.	Gradient CYP (metabolizm) i hERG kolidują: to co jest substratem CYP (aromatyczne, lipofilne) to często bloker hERG. Model nie może jednocześnie dobrze uczyć się obu.
DILI	Drug-Induced Liver Injury — uszkodzenie wątroby przez reaktywne metabolity, stres oksydacyjny, mitochondriotoksyczność.	DILI zależy od globalnych właściwości cząsteczki + reaktywności metabolicznej. 3-hop GIN bez VN nie widzi całości. Gradient z tasków lokalnych (CYP) dominuje.
skin_reaction	Uczulenie kontaktowe — reakcja immunologiczna skóry. Zależy od zdolności cząsteczki do haptenizacji białek.	Silna nierównowaga klas (~15% pozytywnych) + mechanizm niezwiązany z metabolizmem → gradient CYP dominuje, signal skin_reaction tłumiony.

12.3 Wnioski dla projektowania leków (Drug Discovery)

1. **hERG jest krytycznym endpointem wczesnego screeningu.** Kardiotoksyczność przez blokowanie hERG (zespół długiego QT) to jedna z najczęstszych przyczyn wycofania leków z rynku (Terfenadyna, Cisaprid, Astemizol). Model MTL, który pogarsza hERG względem STL, jest **niebezpieczny w zastosowaniu praktycznym**, bo pomija niebezpieczne cząsteczki.
2. **DILI jest najtrudniejszym do przewidzenia endpointem.** Mechanizm hepatotoksyczności jest wieloczynnikowy (reaktywne metabolity, Idiosynkratyczne, dawkozależne), co tłumaczy, dlaczego potrzebna jest globalna informacja (VN) i dlaczego standardowy GIN nie radzi sobie dobrze.
3. **Metabolizm CYP jest relatywnie "łatwy" dla MTL.** 5 izoform CYP dzieli podobną SAR (substraty to lipofilne, aromatyczne cząsteczki) — MTL naturalnie tu pomaga przez transfer wiedzy między podobnymi zadaniami.
4. **Jedno rozwiązanie nie rozwiązuje wszystkich problemów:**
 - skin_reaction → BALANCING (klasy nierównomierne)

- hERG → CROSS_UNCERT (izolacja od CYP-gradientu)
- dili → VIRTUAL_NODE (globalny kontekst molekularny)
- CYP → GROUPING lub PCGRAD
- Globalnie → PCGRAD (stabilna poprawa na wszystkim)

12.4 Potencjalne kierunki dalszych badań

- **Ensemble PCGrad + Virtual Node:** PCGrad naprawia konflikty gradientów, VN dostarcza globalnego kontekstu. Oba działają na ortogonalnych poziomach (optymalizacja vs. reprezentacja) — ich połączenie może być addytywne.
- **Hierarchiczne MTL:** zamiast flat MTL, model drzewiasty: backbone → gałąź CYP → 5 głowic CYP; backbone → gałąź kardio/toksyczność → hERG/DILI/etc. Redukuje konflikty gradientów strukturalnie.
- **Pretraining na dużych bazach danych:** ChEMBL/PubChem pretraining GNN na self-supervised tasks (masking atomów, predykcja właściwości) może nauczyć globalnych reprezentacji zanim zacznie się fine-tuning na ADMET.
- **Attention-based pooling zamiast global_add_pool:** Zamiast sumy, nauczony mechanizm attention może skoncentrować się na różnych fragmentach cząsteczki per task (CYP: miejsce aktywne; hERG: fragment aromatyczny; BBB: polarność całości).
- **Task-specific augmentacja danych:** `skin_reaction` i `dili` mają mało danych — data augmentation (konformalnie podobne cząsteczki, SMILES augmentation przez randomizację kolejności atomów) mogłoby wyrównać dysproporcję.

```
In [15]: # Podsumowanie końcowe – tabela wniosków per zadanie
summary_data = {
    'hia_hou': ('Absorption', 'VIRTUAL_NODE', 'GNN_RDKIT_MORGAN', 'Globalna lipofilność/MW → VN pomaga'),
    'pgp_broccatelli': ('Absorption', 'GROUPING', 'GNN_RDKIT_MORGAN', 'Transport aktywny P-gp – shared CYP SAR'),
    'bbb_martins': ('Absorption', 'PCGRAD', 'GNN_RDKIT_MORGAN', 'BBB zależy od globalnych właściwości'),
    'cyp2c19_veith': ('Metabolism', 'MAIN', 'GNN_RDKIT_MORGAN', 'CYP – lokalna SAR, MTL natural'),
    'cyp2d6_veith': ('Metabolism', 'MAIN', 'GNN_RDKIT_MORGAN', 'CYP – najbardziej specyficzny, baseline wystarczy'),
    'cyp3a4_veith': ('Metabolism', 'GROUPING', 'GNN_RDKIT_MORGAN', 'CYP3A4 ~50% leków – izolacja backboneu pomaga'),
    'cyp1a2_veith': ('Metabolism', 'GROUPING', 'GNN_RDKIT_MORGAN', 'CYP1A2 metabolizuje aromatyczne aminy'),
    'cyp2c9_veith': ('Metabolism', 'GROUPING', 'GNN_RDKIT_MORGAN', 'CYP2C9 – warfaryna/NLPZ'),
    'herg': ('Toxicity', 'CROSS_UNCERT', 'GNN_RDKIT_MORGAN', 'Kardiotoks: izolacja od CYP-gradientu kluczowa'),
    'dili': ('Toxicity', 'VIRTUAL_NODE', 'GNN_RDKIT_MORGAN', 'Hepatotoks: globalna reaktywność cząsteczki'),
    'skin_reaction': ('Toxicity', 'BALANCING', 'GNN_RDKIT_MORGAN', 'Uczulenie kontaktowe: nierówne klasy'),
    'ames': ('Toxicity', 'PCGRAD', 'GNN_RDKIT_MORGAN', 'Genotoksyczność Ames: gradient surgery stabilizuje'),
    'clintox': ('Toxicity', 'PCGRAD', 'GNN_RDKIT_MORGAN', 'Toksyczność kliniczna: PCGRAD wygrywa wyraźnie'),
}
```

```
sdf = pd.DataFrame(summary_data, index=['Grupa ADMET', 'Najlepsza metoda', 'Wariant', 'Interpretacja']).T
sdf.index.name = 'Task'

GROUP_COLORS = {'Absorption': '#dce9f7', 'Metabolism': '#fef3cd', 'Toxicity': '#fde8e8'}

def hl_group(val):
    return f'background-color: {GROUP_COLORS.get(val, "")}'

def hl_method(val):
    c = METHOD_COLORS.get(val, '')
    if c:
        return f'color: {c}; font-weight: bold'
    return ''

display(sdf.style
        .applymap(hl_group, subset=['Grupa ADMET'])
        .applymap(hl_method, subset=['Najlepsza metoda'])
        .set_caption('Końcowe wnioski per zadanie – mechanizm biologiczny × optymalna metoda MTL')
        .set_properties(**{'font-size': '10.5pt', 'text-align': 'left'})
        )
```

Końcowe wnioski per zadanie — mechanizm biologiczny × optymalna metoda MTL

Task	Grupa ADMET	Najlepsza metoda	Wariant	Interpretacja
hia_hou	Absorption	VIRTUAL_NODE	GNN_RDKIT_MORGAN	Globalna lipofilność/MW → VN pomaga
pgp_broccatelli	Absorption	GROUPING	GNN_RDKIT_MORGAN	Transport aktywny P-gp — shared CYP SAR
bbb_martins	Absorption	PCGRAD	GNN_RDKIT_MORGAN	BBB zależy od globalnych właściwości
cyp2c19_veith	Metabolism	MAIN	GNN_RDKIT_MORGAN	CYP — lokalna SAR, MTL natural
cyp2d6_veith	Metabolism	MAIN	GNN_RDKIT_MORGAN	CYP — najbardziej specyficzny, baseline wystarczy
cyp3a4_veith	Metabolism	GROUPING	GNN_RDKIT_MORGAN	CYP3A4 ~50% leków — izolacja backbonu pomaga
cyp1a2_veith	Metabolism	GROUPING	GNN_RDKIT_MORGAN	CYP1A2 metabolizuje aromatyczne aminy
cyp2c9_veith	Metabolism	GROUPING	GNN_RDKIT_MORGAN	CYP2C9 — warfaryna/NLPZ
herg	Toxicity	CROSS_UNCERT	GNN_RDKIT_MORGAN	Kardiotoks: izolacja od CYP-gradientu kluczowa
dili	Toxicity	VIRTUAL_NODE	GNN_RDKIT_MORGAN	Hepatotoks: globalna reaktywność cząsteczki
skin_reaction	Toxicity	BALANCING	GNN_RDKIT_MORGAN	Uczulenie kontaktowe: nierówne klasy
ames	Toxicity	PCGRAD	GNN_RDKIT_MORGAN	Genotoksyczność Ames: gradient surgery stabilizuje
clintox	Toxicity	PCGRAD	GNN_RDKIT_MORGAN	Toksyczność kliniczna: PCGRAD wygrywa wyraźnie